TRAFFIC SIGN CLASSIFICATION REPORT

Major Project Report

Submitted by:
Bhumika Johri

1. Introduction

Traffic sign recognition is an important application of computer vision used in autonomous vehicles and driver assistance systems. The objective of this project is to develop a deep learning model that can accurately classify traffic signs into multiple categories using Convolutional Neural Networks (CNN). This system helps in understanding road signs automatically, which is essential for intelligent transportation systems and robotics applications.

2. Problem Statement

To design and implement a CNN-based image classification model that can classify traffic sign images into 43 different categories with high accuracy.

3. Dataset Description

The dataset used in this project is the GTSRB (German Traffic Sign Recognition Benchmark) dataset.

- Total Classes: 43
- Colored traffic sign images
- Images resized to 30 × 30 pixels

Each class represents a different traffic sign such as speed limits, warning signs, and stop signs.

4. Data Preprocessing

The following preprocessing steps were performed:

- Resized all images to 30 × 30 pixels
- Converted images into NumPy arrays
- Normalized pixel values by dividing by 255
- Converted labels into categorical format
- Split dataset into 80% training and 20% testing

These steps ensure better model performance and faster convergence.

5. Model Architecture

The Deep Convolutional Neural Network (CNN) consists of:

- Conv2D (32 filters) + Batch Normalization + MaxPooling
- Conv2D (64 filters) + Batch Normalization + MaxPooling
- Conv2D (128 filters) + Batch Normalization + MaxPooling
- Flatten Layer
- Dense Layer (256 neurons)
- Dropout Layer (0.5)
- Output Layer (43 neurons with Softmax activation)

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 28, 28, 32)	896
batch_normalization (BatchNormalization)	(None, 28, 28, 32)	128
max_pooling2d (MaxPooling2D)	(None, 14, 14, 32)	0
conv2d_1 (Conv2D)	(None, 12, 12, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 12, 12, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 4, 4, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten (Flatten)	(None, 512)	0

(BatchNormalization)		
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 4, 4, 128)	512
max_pooling2d_2 (MaxPooling2D)	(None, 2, 2, 128)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 256)	131,328
dropout (Dropout)	(None, 256)	0
dense_1 (Dense)	(None, 43)	11,051

Total params: 236,523 (923.92 KB)
Trainable params: 236,075 (922.17 KB)
Non-trainable params: 448 (1.75 KB)

Convolution layers extract features, Batch Normalization improves stability, MaxPooling reduces dimensions, Dropout prevents overfitting, and Softmax performs multi-class classification.

6. Model Training

Optimizer: Adam
Loss Function: Categorical Crossentropy
Batch Size: 32
Epochs: 10

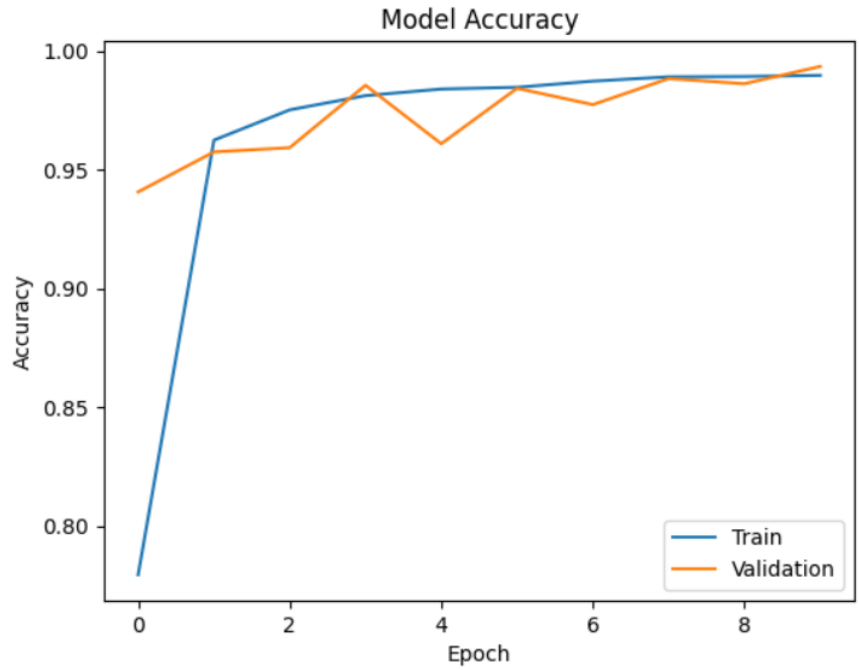
The model was trained on the training dataset and validated on the testing dataset.

```
Total params: 236,523 (923.92 KB)
Trainable params: 236,075 (922.17 KB)
Non-trainable params: 448 (1.75 KB)
Epoch 1/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 40s 39ms/step - accuracy: 0.5784 - loss: 1.6265 - val_accuracy: 0.9406 - val_loss: 0.1953
Epoch 2/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 38ms/step - accuracy: 0.9567 - loss: 0.1360 - val_accuracy: 0.9574 - val_loss: 0.1477
Epoch 3/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 37ms/step - accuracy: 0.9747 - loss: 0.0834 - val_accuracy: 0.9592 - val_loss: 0.1437
Epoch 4/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 36s 37ms/step - accuracy: 0.9826 - loss: 0.0579 - val_accuracy: 0.9855 - val_loss: 0.0500
Epoch 5/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 36s 37ms/step - accuracy: 0.9844 - loss: 0.0512 - val_accuracy: 0.9609 - val_loss: 0.1266
Epoch 6/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 37s 37ms/step - accuracy: 0.9845 - loss: 0.0509 - val_accuracy: 0.9842 - val_loss: 0.0493
Epoch 7/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 38ms/step - accuracy: 0.9883 - loss: 0.0378 - val_accuracy: 0.9773 - val_loss: 0.0815
Epoch 8/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 41s 39ms/step - accuracy: 0.9876 - loss: 0.0433 - val_accuracy: 0.9883 - val_loss: 0.0441
Epoch 9/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 41s 38ms/step - accuracy: 0.9900 - loss: 0.0350 - val_accuracy: 0.9861 - val_loss: 0.0481
Epoch 10/10
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 39ms/step - accuracy: 0.9892 - loss: 0.0400 - val_accuracy: 0.9934 - val_loss: 0.0258
```

7. Results

Training Accuracy: ~99%
Validation Accuracy: 98%

```
981/981 ━━━━━━━━━━━━━━━━━━━━ 38s 39ms/step - accuracy: 0.9892 - loss: 0.0400 - val_accuracy: (
```



The accuracy graph shows stable learning and good generalization performance, demonstrating the effectiveness of the CNN model.

8. Applications

- Self-driving cars
- Driver assistance systems
- Smart traffic monitoring
- Robotics navigation systems

9. Conclusion

A deep CNN-based model was successfully developed to classify traffic signs into 43 categories. The model achieved 98% validation accuracy, indicating strong performance and reliability. This project demonstrates

practical implementation of computer vision and deep learning techniques.

10. Future Improvements

- Use Transfer Learning (MobileNet/ResNet)
- Deploy as a web application
- Implement real-time detection using live camera